# Apuntes HTML y CSS

*Material de referencia para acompañar la Clase 02 y los ejercicios prácticos.*
*Cubre los conceptos necesarios para el examen parcial.*

## Índice

**Requisitos** ..... 2
**Cómo Funciona la Web** ..... 3
URL (Uniform Resource Locator) ..... 3
Peticiones HTTP ..... 4
URLs internas y externas ..... 4
URLs absolutas y relativas ..... 5
**HTML** ..... 6
**Introducción a HTML** ..... 6
¿Qué es HTML? ..... 6
Elementos HTML ..... 6
Atributos ..... 7
Tipos de elementos: bloque vs. línea ..... 7
**Estructura de un documento HTML5** ..... 7
**Etiquetas HTML más usadas** ..... 10
**Links y navegación** ..... 11
**Imágenes** ..... 11
**Listas** ..... 11
**Tablas** ..... 12
**Formularios** ..... 13
**Convenciones y buenas prácticas HTML** ..... 15
**CSS** ..... 16
**Introducción a CSS** ..... 16
¿Qué es CSS? ..... 16
Reglas CSS ..... 16
Selectores CSS ..... 17
Cómo incluir CSS en HTML ..... 19
**El modelo de cajas (Box Model)** ..... 20
**Propiedades CSS básicas** ..... 23
**Colores y unidades de medida** ..... 24
**Variables CSS (Custom Properties)** ..... 25
**Flexbox** ..... 26
**Diseño Responsive y Media Queries** ..... 29

# Requisitos

## Navegador Web

Para poder visualizar los ejemplos y ejercicios que escribiremos, es necesario un Navegador Web, si bien es posible utilizar cualquiera, recomiendo el uso de Google Chrome o Firefox, ya que nos brinda además herramientas extra para trabajar y unificar criterios.

## Editores de Código / IDEs

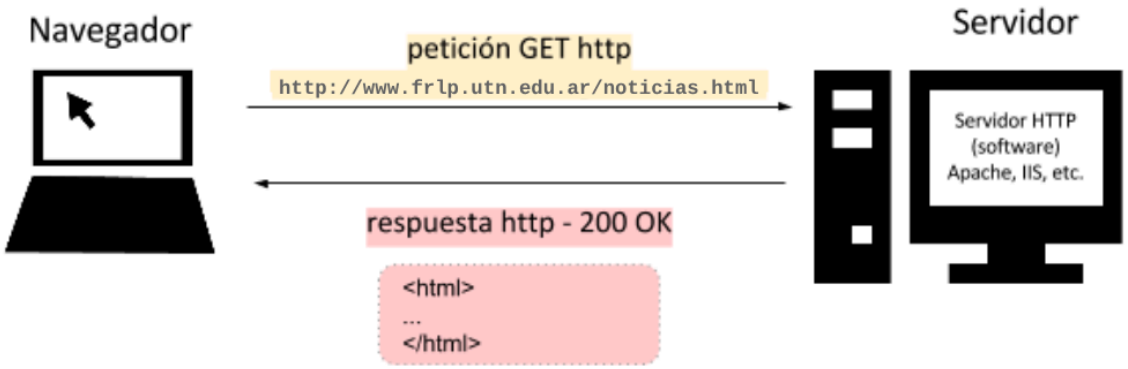
Si bien el código puede escribirse en cualquier editor de texto plano, se recomienda el uso de editores de texto que reconocen el código de los lenguajes utilizados, que además del aspecto visual, nos brindan funciones y herramientas específicas para facilitar el armado de código de calidad.

- VSCode
- Sublime Text
- Cualquier otro

## Cómo Funciona la Web

Cuando abrís una página web, pasan cuatro cosas:

1. Escribís una **URL** en el navegador (ej: https://www.google.com).
2. El navegador envía una **petición HTTP** al servidor que aloja esa página.
3. El servidor responde con **archivos**: HTML (estructura), CSS (estilos), y opcionalmente JavaScript (interacción).
4. El navegador **interpreta** (*parsea*) esos archivos y los muestra en pantalla (*renderiza*).




## URL (Uniform Resource Locator)

Cada recurso en internet tiene una dirección única. El formato general es:

**`[scheme://[user:password@]host[:port]][/]path[?query][#fragment]`**

*Los corchetes [ ] indican que lo que se encuentra contenido entre ellos es opcional.*

Para la web, en general utilizaremos este formato:

```
protocolo://servidor[:puerto]/ruta[?consulta][#seccion]
```

**Ejemplo**:

```
https://www.frlp.utn.edu.ar/seccion/novedades.html?page=3&limit=20#comentarios
```

| Parte | Ejemplo | Descripción |
|---|---|---|
| Protocolo | `https://` | Cómo se comunican (HTTP o HTTPS) |
| Servidor (host) | `www.frlp.utn.edu.ar` | Máquina que tiene los archivos |
| Puerto | `:443` *(default HTTPS)* | Punto de entrada (generalmente implícito) |
| Ruta | `/seccion/novedades.html` | Ubicación del recurso en el servidor |
| Consulta | `?page=3` | Parámetros adicionales |
| Sección | `#comentarios` | Posición dentro de la página |

## Peticiones HTTP

El navegador se comunica con el servidor mediante el protocolo HTTP. Los dos tipos de petición que utilizaremos por ahora son:

- **GET** para solicitar información (cargar una página, pedir datos)
- **POST** para enviar información (enviar un formulario, subir datos)

El servidor responde con un **código de estado**:

| Código | Significado |
|---|---|
| 200 | OK — recurso encontrado |
| 301 | Movido permanentemente (redirección) |
| 403 | Prohibido — no tenés permiso |
| 404 | No encontrado — la ruta no existe |
| 500 | Error interno del servidor |

*En el Bloque 3 se verá con mayor detalle el protocolo HTTP.*

## URLs internas y externas

- **Externas**: apuntan a otro sitio (`https://www.google.com`)
- **Internas**: apuntan a otra página del mismo sitio (`/contacto.html`)

## URLs absolutas y relativas

- **Absoluta**: incluye protocolo + servidor + ruta
  **https://mi-sitio.com/paginas/contacto.html**
- **Relativa**: omite las partes comunes, se resuelve desde la página actual:
  - `contacto.html` → en el mismo directorio
  - `../index.html` → un nivel arriba
  - `/paginas/contacto.html` → desde la raíz del sitio

Las relativas son más prácticas para enlaces internos: si cambia el dominio, los enlaces siguen funcionando.

# HTML

## Introducción a HTML

### ¿Qué es HTML?

HTML (HyperText Markup Language) es el lenguaje que define **la estructura y el contenido** de las páginas web. No es un lenguaje de programación: es un lenguaje de **marcado**, que usa etiquetas para indicar qué es cada parte del documento.

Es un estándar universal: el mismo HTML se ve de forma similar en cualquier navegador y sistema operativo.

### Elementos HTML

Un documento HTML está compuesto por **elementos**. Cada elemento se escribe con etiquetas: `<etiqueta>[Contenido]</etiqueta>`

Los elementos, según su sintaxis (cómo se escriben), se pueden clasificar en dos clases:

**Elemento tradicional (con contenido):**

Están formados por un **par de tags**, uno de **apertura** (elemento entre <>, puede contener atributos) y otro de **cierre** (similar al de apertura, antepone una / al elemento).

El **contenido** puede ser el texto u otros Elementos HTML que están contenidos dentro del Elemento. Es la parte que se muestra al usuario final.

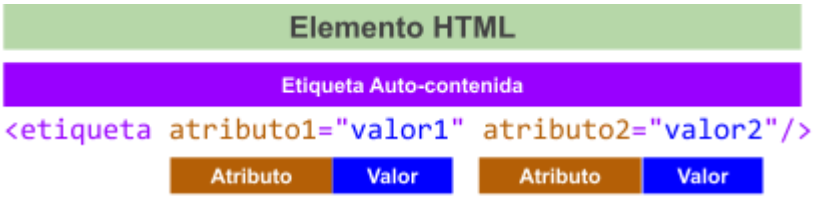
```
<p>Este es un párrafo de texto.</p>
```

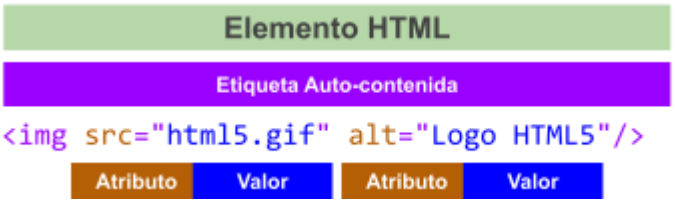


**Elemento auto-contenido (sin contenido):**

Constan sólo de **un único tag**, esto se debe a que **no cuentan con contenido** adjunto explícito. Es similar a la etiqueta de apertura de los elementos típicos, a diferencia que se pueden escribir con una "/" al final.

```
<img src="foto.jpg" alt="Descripción de la foto"/>
<br> <br/>
<input type="text">
```




**Atributos**

Las etiquetas pueden tener **atributos** que agregan información o configuran el comportamiento del elemento:

```
<a href="https://google.com" target="_blank">Ir a Google</a>
<img src="logo.png" alt="Logo del sitio" width="200">
```

Los atributos van siempre en la etiqueta de apertura, con el formato: `nombre="valor"`

**Tipos de elementos: bloque vs. línea**

- **Bloque** (*block*): ocupan todo el ancho disponible y empiezan en nueva línea. Ejemplos: `<p>`, `<div>`, `<h1>`, `<section>`
- **Línea** (*inline*): ocupan solo el espacio de su contenido, no generan salto de línea. Ejemplos: `<a>`, `<span>`, `<strong>`, `<em>`, `<img>`

## Estructura de un documento HTML5

Todo documento HTML5 tiene esta estructura base:

```html
<!DOCTYPE html>
<html lang="es">
 <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Título de la página</title>
  <link rel="stylesheet" href="styles.css">
 </head>
 <body>
  <header>
    <h1>Mi Sitio</h1>
    <nav>
      <a href="index.html">Inicio</a>
      <a href="contacto.html">Contacto</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Sección principal</h2>
      <p>Contenido de la página.</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Mi Sitio</p>
  </footer>
</body>
</html>
```

| Elemento | Función |
|---|---|
| `<!DOCTYPE html>` | Indica que es HTML5 |
| `<html lang="es">` | Raíz del documento, idioma español |
| `<head>` | Metadata (no visible al usuario) |
| `<meta charset="UTF-8">` | Codificación para acentos y ñ |
| `<meta name="viewport" ...>` | Que se vea bien en celulares |
| `<title>` | Título en la pestaña del navegador |
| `<link rel="stylesheet">` | Vincula la hoja de estilos CSS |
| `<body>` | Todo el contenido visible |

### Etiquetas semánticas HTML5

HTML5 introdujo etiquetas que **describen el significado** de cada parte de la página, en lugar de usar `<div>` genéricos:

| Etiqueta | Uso |
|---|---|
| `<header>` | Cabecera de la página o sección |
| `<nav>` | Navegación principal |
| `<main>` | Contenido principal (uno solo por página) |
| `<section>` | Sección temática |
| `<article>` | Contenido independiente (un post, una noticia) |
| `<aside>` | Contenido lateral o relacionado |
| `<footer>` | Pie de página o sección |

**¿Por qué importa?**

- **Accesibilidad**: los lectores de pantalla entienden la estructura
- **SEO**: los buscadores indexan mejor el contenido
- **Mantenimiento**: el código es más claro para vos y tu equipo

**Regla general:** si existe una etiqueta semántica para lo que necesitás, usala en lugar de `<div>`.

## Etiquetas HTML más usadas

### Títulos

```
<h1>Título principal</h1>    <!-- Solo uno por página -->
<h2>Subtítulo</h2>
<h3>Sub-subtítulo</h3>
<!-- ... hasta <h6> -->
```

### Párrafos y texto

```
<p>Un párrafo de texto.</p>
<strong>Texto importante (negrita semántica)</strong>
<em>Texto enfatizado (itálica semántica)</em>
<br/>   <!-- Salto de línea -->
<hr/>   <!-- Línea horizontal divisoria -->
```

### Contenedores genéricos

```
<div>Contenedor de bloque (para agrupar elementos)</div>
<span>Contenedor en línea (para estilar texto dentro de un párrafo)</span>
```

### Comentarios

```
<!-- Esto es un comentario, no se muestra en el navegador -->
```

## Links y navegación

```html
<!-- Link externo -->
<a href="https://www.google.com">Ir a Google</a>

<!-- Link externo que abre en nueva pestaña -->
<a href="https://www.google.com" target="_blank">Google (nueva pestaña)</a>

<!-- Link interno (otra página del mismo sitio) -->
<a href="contacto.html">Contacto</a>

<!-- Link a una sección dentro de la misma página (por id) -->
<a href="#seccion-2">Ir a Sección 2</a>
<!-- ... más abajo ... -->
<h2 id="seccion-2">Sección 2</h2>
```

## Imágenes

```html
<img src="imagenes/foto.jpg" alt="Descripción de la foto">
```

| Atributo | Obligatorio | Descripción |
|---|---|---|
| `src` | Sí | Ruta de la imagen (relativa o absoluta) |
| `alt` | Recomendado | Texto alternativo (accesibilidad y SEO) |
| `width` | No | Ancho en píxeles |
| `height` | No | Alto en píxeles |

**Buena práctica**: siempre incluir el atributo `alt`. Si la imagen es decorativa, usar `alt=""`.

## Listas

### Lista sin orden (viñetas)

```html
<ul>
  <li>Primer ítem</li>
  <li>Segundo ítem</li>
  <li>Tercer ítem</li>
</ul>
```

**Lista ordenada (números)**

```
<ol>
  <li>Paso 1</li>
  <li>Paso 2</li>
  <li>Paso 3</li>
</ol>
```

Las listas se pueden anidar (una lista dentro de un `<li>`).

## Tablas

Las tablas se usan para datos tabulares (**NO** para maquetación).

```
<table>
 <tr>
   <td>Fila 1, Columna 1</td>
   <td>Fila 1, Columna 2</td>
 </tr>
 <tr>
   <td>Fila 2, Columna 1</td>
   <td>Fila 2, Columna 2</td>
 </tr>
</table>
```

| Etiqueta | Función |
|---|---|
| `<table>` | Contenedor de la tabla |
| `<tr>` | Fila (table row) |
| `<td>` | Celda (table data) |
| `<th>` | Celda de encabezado (table header) — se muestra en negrita |

Atributos útiles en `<td>`: `colspan="2"` (une celdas horizontalmente), `rowspan="2"` (une celdas verticalmente).

## Formularios

Los formularios permiten que el usuario **ingrese y envíe datos**. Se usan en login, registro, búsquedas, formularios de contacto, filtros, encuestas — cualquier interacción donde el usuario aporta información.

### Estructura de un formulario

```
<form>
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre"
         placeholder="Tu nombre" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email"
         placeholder="email@ejemplo.com" required>

  <label for="password">Contraseña:</label>
  <input type="password" id="password" name="password"
         minlength="6" required>

  <label for="mensaje">Mensaje:</label>
  <textarea id="mensaje" name="mensaje" rows="4"></textarea>

  <button type="submit">Enviar</button>
</form>
```

### Elementos clave

| Elemento | Función |
|---|---|
| `<form>` | Envuelve todos los campos del formulario |
| `<label for="...">` | Describe un campo. El for debe coincidir con el id del input |
| `<input type="...">` | Campo de entrada. El type define qué tipo de dato acepta |
| `<textarea>` | Campo de texto largo (multilínea) |
| `<select> + <option>` | Lista desplegable |
| `<button type="submit">` | Botón para enviar el formulario |

**Ejemplo: select (lista desplegable)**

```
<label for="tema">Tema favorito:</label>
<select id="tema" name="tema">
  <option value="">-- Elegir --</option>
  <option value="ficcion">Ficción</option>
  <option value="historia">Historia</option>
  <option value="ciencia">Ciencia</option>
</select>
```

**Tipos de input más comunes**

| Type | Muestra | Validación nativa |
|---|---|---|
| `text` | Campo de texto | — |
| `email` | Campo de texto | Verifica formato email |
| `password` | Campo con puntos | Oculta el texto |
| `number` | Campo numérico | Solo acepta números, soporta `min`/`max` |
| `date` | Selector de fecha | Formato de fecha |
| `tel` | Campo de teléfono | — (formato libre) |
| `file` | Selector de archivo | Permite `accept="image/*"` |
| `checkbox` | Casilla de verificación | — |
| `radio` | Opción única entre varias | Mismo `name` agrupa las opciones |

### Atributos de validación HTML5

| Atributo | Función |
|---|---|
| `required` | El campo no puede estar vacío |
| `minlength="6"` | Mínimo de caracteres |
| `maxlength="100"` | Máximo de caracteres |
| `min="1" / max="99"` | Valor mínimo/máximo (para number, date) |
| `pattern="[A-Za-z]+"` | Expresión regular que debe cumplir el valor |
| `placeholder="texto"` | Texto de ejemplo dentro del campo (no es un valor) |

Estas validaciones las hace el **navegador automáticamente**, sin necesidad de escribir código.

## Convenciones y buenas prácticas HTML

### En el código

- Las etiquetas se cierran en el **orden inverso** a como se abren:

```
<!-- MAL -->  <p>Texto <b>destacado</p></b>
<!-- BIEN --> <p>Texto <b>destacado</b></p>
```

- Nombres de etiquetas y atributos siempre en **minúsculas**
- Valores de atributos siempre entre **comillas dobles**:

```
<!-- MAL -->  <img src='foto.jpg' alt='Logo'>
<!-- BIEN --> <img src="foto.jpg" alt="Logo">
```

- Indentar consistentemente (2 o 4 espacios por nivel)
- Siempre incluir `<!DOCTYPE html>` y `lang="es"`
- Siempre usar `charset="UTF-8"`
- Evitar estilos en línea (atributo `style`)

### Nombres de archivos

- En minúsculas: `contacto.html`, **NO**: Contacto.HTML
- Sin acentos ni caracteres especiales
- Usar guiones en vez de espacios: `mi-pagina.html`
- La página principal siempre se llama `index.html`

# CSS

## Introducción a CSS

### ¿Qué es CSS?

CSS (Cascading Style Sheets) es el lenguaje de marcado que controla la **presentación visual** de un documento HTML: colores, tipografía, espaciado, disposición de los elementos en pantalla, etc.

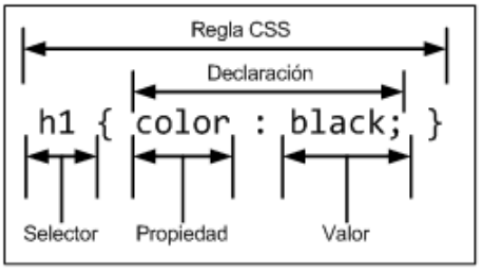
La idea fundamental es la **separación de responsabilidades**:

- **HTML** → **qué** mostramos (estructura y contenido)
- **CSS** → **cómo** se ve (presentación y estilos)

### Reglas CSS

Una regla CSS se compone de un **selector** (a quién aplicar) y una **declaración** (qué aplicar):

```
selector {
   propiedad: valor;
   propiedad: valor;
    ...
}
```




**Ejemplo**

```
h1 {
    color: blue;
    font-size: 2rem;
}
```

## Selectores CSS

Los selectores le dicen a CSS **a qué elementos** aplicar los estilos.

### Selectores básicos

| Selector | Sintaxis | Aplica a... |
|---|---|---|
| Universal | `* { }` | Todos los elementos |
| De etiqueta | `p { }` | Todos los <p> |
| De clase | `.destacado { }` | Elementos con `class="destacado"` |
| De ID | `#titulo { }` | El elemento con `id="titulo"` |

```html
<p class="destacado">Texto destacado</p>
<p>Texto normal</p>
<p id="intro">Introducción</p>
```

```css
.destacado { color: red; }
#intro { font-weight: bold; }
p { font-size: 1rem; }
```

**El selector de clase (`.clase`) es el más usado**. Permite reutilizar estilos en múltiples elementos.

### Combinar selectores

Utilizar más de un selector unidos sin espacios intermedios, permite seleccionar elementos de manera más específica.

```css
/* Solo los <span> que tengan la clase “destacado” */
span.destacado {
  color: red;
}
```

### Selector descendente

Aplica a elementos que están **dentro** de otros:

```css
/* Solo los <a> que están dentro de <nav> */
nav a {
  text-decoration: none;
  color: white;
}
```

### Agrupar selectores

Se pueden aplicar las mismas reglas a varios selectores separándolos con **coma**:

```css
/* todos los <h1>, <h2> o <h3>  */
h1, h2, h3 {
  font-family: Arial, sans-serif;
}
```

### Especificidad (resolución de conflictos)

Cuando varias reglas aplican al mismo elemento, gana la más específica:

1. Estilos en línea *[evitar]* *(mayor prioridad)*
2. Selector de ID (`#id`)
3. Selector de clase (`.clase`)
4. Selector de etiqueta (`p`)
5. Selector universal (`*`) *(menor prioridad)*

A igual especificidad, gana la última regla definida.

### Herencia

Muchas propiedades CSS se **heredan** de padres a hijos. Si definís `color: blue` en el `<body>`, todo el texto de la página será azul, salvo que un elemento hijo lo sobreescriba.

## Cómo incluir CSS en HTML

**Archivo externo** *(**RECOMENDADO**: siempre usar este)*

Crear un archivo `.css` y vincularlo en el `<head>` del HTML:

```html
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

**Ventajas**: un solo archivo CSS controla los estilos de todas las páginas del sitio. Cambiar un valor afecta todo de forma consistente.

**Estilos internos** *(**Evitar** salvo para prototipos rápidos)*

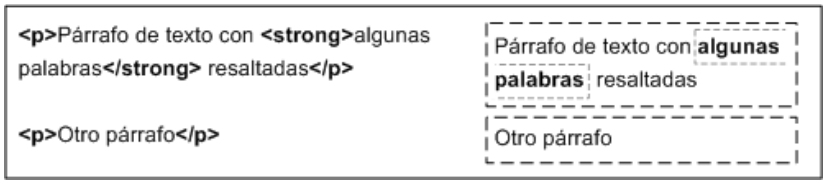
```html
<head>
  <style>
    h1 { color: navy; }
  </style>
</head>
```

**Estilos en línea** *(**NO USAR**, mala práctica)*

```html
<p style="color: red;">Texto rojo</p>
```

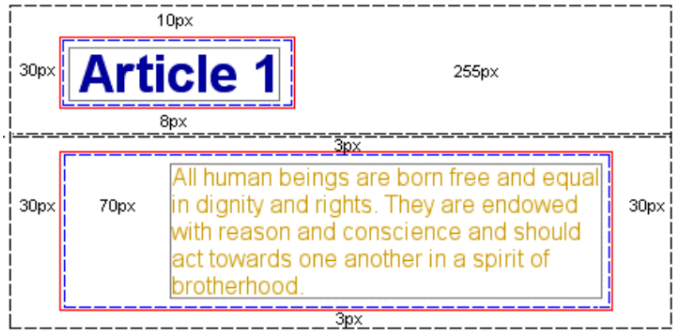
## El modelo de cajas (Box Model)

Cada elemento HTML se representa como una **caja rectangular**. El Box Model define cómo se calcula el espacio que ocupa.

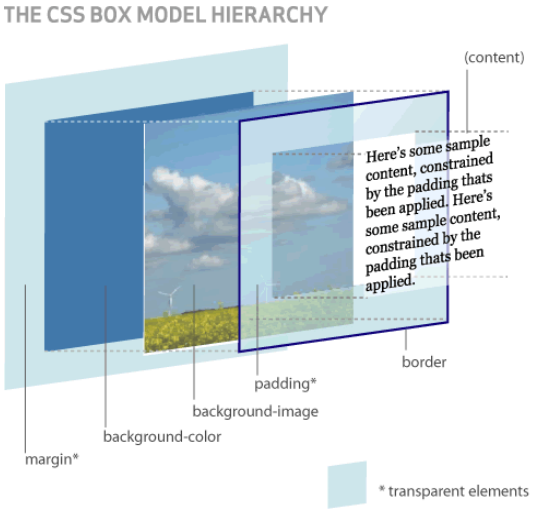


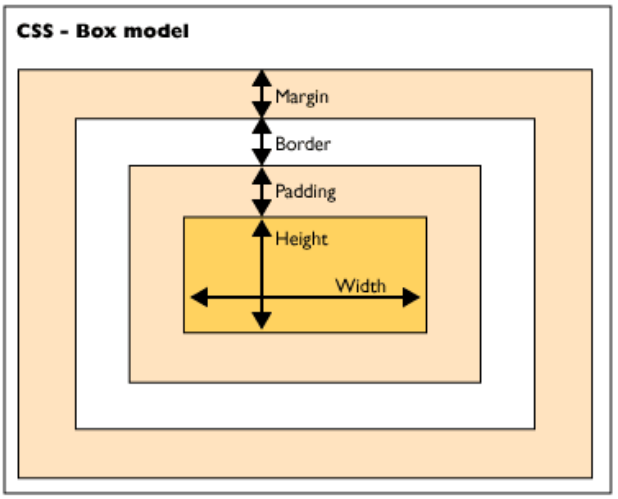

La imagen muestra tres cajas rectangulares que crean las tres etiquetas HTML que incluye la página.

Los navegadores crean y colocan las cajas de forma automática, pero CSS permite modificar todas sus características.




Cada una de las cajas está formada por cuaro partes, tal y como muestra la siguiente imagen:




Las partes que componen cada caja y su orden de visualización desde el punto de vista del usuario son las siguientes:

| Parte | Descripción |
|---|---|
| `content` *(contenido)* | El contenido del elemento (texto, imagen, etc) |
| `padding` *(relleno)* | Espacio interno entre el contenido y el borde |
| `border` *(borde)* | Línea que rodea el padding |
| `margin` *(margen)* | Espacio externo entre este elemento y los demás |

**El problema del ancho por defecto**

Por defecto, `width` aplica solo al contenido. El `padding` y border se `suman` encima:

```css
/* Esto hace que la caja ocupe 200 + 20 + 20 + 2 + 2 = 244px */
.caja {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
}
```

**La solución: box-sizing**

Para que `width` incluya `padding` y `border`, se usa `box-sizing: border-box`. Se recomienda aplicarlo globalmente:

```css
* {
  box-sizing: border-box;
}
```

Con esto, si decís `width: 200px`, la caja ocupa exactamente 200px, y el **padding/border se descuentan del espacio interior.**

### Propiedades del Box Model

```css
.elemento {
  /* Margin (espacio externo) */
  margin: 1rem;                  /* todos los lados */
  margin: 1rem 2rem;             /* vertical horizontal */
  margin: 1rem 2rem 1rem 2rem;   /* top right bottom left */

  /* Padding (espacio interno) */
  padding: 1rem;
  padding-top: 0.5rem;

  /* Border */
  border: 1px solid #ccc;
  border-radius: 8px;            /* esquinas redondeadas */

  /* Dimensiones */
  width: 300px;
  max-width: 100%;               /* no exceder el contenedor */
  height: auto;                  /* se ajusta al contenido */
}
```

## Propiedades CSS básicas

### Texto y tipografía

```css
.texto {
  color: #333;                    /* color del texto */
  font-family: Arial, sans-serif; /* tipografía (con fallback) */
  font-size: 1rem;                /* tamaño */
  font-weight: bold;              /* grosor (normal, bold, 400, 700) */
  font-style: italic;             /* estilo */
  text-align: center;        /* alineación (left, right, center, justify) */
  text-decoration: underline;     /* subrayado, tachado, ninguno */
  text-transform: uppercase;      /* mayúsculas, minúsculas */
  line-height: 1.5;               /* interlineado */
}
```

### Fondos y sombras

```css
.fondo {
  background-color: #f5f5f5;
  background-image: url("fondo.jpg");
  background-repeat: no-repeat;
  background-size: cover;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);  /* sombra */
}
```

### Bordes

```css
.borde {
  border: 1px solid #ccc;
  border-radius: 8px;       /* esquinas redondeadas */
  border-bottom: 2px solid blue;
}
```

### Visualización

```css
.visibilidad {
  display: block;       /* bloque (ocupa todo el ancho) */
  display: inline;      /* en línea (solo su contenido) */
  display: none;        /* oculta el elemento completamente */
  visibility: hidden;   /* invisible pero ocupa espacio */
  overflow: hidden;     /* oculta contenido que se desborda */
  overflow: auto;       /* agrega scroll si se desborda */
}
```

## Colores y unidades de medida

### Colores

| Formato | Ejemplo | Descripción |
|---|---|---|
| Palabra clave | `red`, `blue`, `white` | Nombres predefinidos (limitados) |
| Hexadecimal | `#ff0000`, `#f00` | 6 o 3 dígitos hex (R, G, B) |
| RGB | `rgb(255, 0, 0)` | Rojo, Verde, Azul (0-255) |
| RGBA | `rgba(255, 0, 0, 0.5)` | RGB + transparencia (0-1) |

Negro: `#000000` / `rgb(0,0,0)` - Blanco: `#ffffff` / `rgb(255,255,255)`

### Unidades de medida

| Unidad | Tipo | Descripción |
|---|---|---|
| `px` | Absoluta | Píxeles. Fija, predecible |
| `%` | Relativa | Porcentaje del elemento padre |
| `em` | Relativa | Relativa al font-size del elemento padre |
| `rem` | Relativa | Relativa al font-size del `<html>` (generalmente 16px) |
| `vw` / `vh` | Relativa | Porcentaje del ancho/alto de la ventana |

**Recomendación:** usar `rem` para tipografía y espaciado, `%` o `vw` para anchos, `px` solo para bordes y valores muy pequeños.

## Variables CSS (Custom Properties)

Permiten definir valores reutilizables. Se definen en `:root` (alcance global) y se usan con `var()`:

```css
:root {
  --primary-color: #3498db;
  --secondary-color: #2c3e50;
  --background-color: #f5f5f5;
  --border-radius: 8px;
  --shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

button {
  background: var(--primary-color);
  border-radius: var(--border-radius);
  box-shadow: var(--shadow);
  color: white;
}

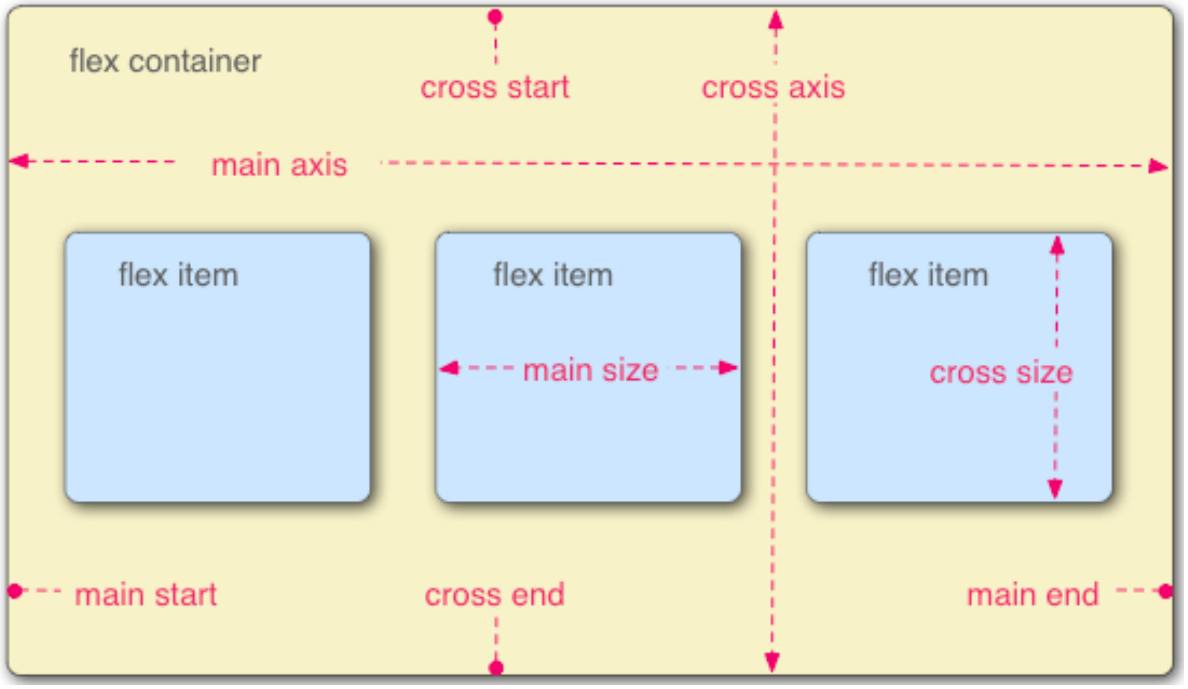
.header {
  background: var(--secondary-color);
}
```

**Ventajas:**

- **Mantenimiento**: cambiás un color en un solo lugar y se actualiza en todo el sitio
- **Consistencia**: todos los elementos usan los mismos valores
- **Temas**: podés cambiar variables para crear un modo oscuro/claro

## Flexbox

Flexbox es la herramienta moderna para crear **layouts** (disposición de elementos en la página). Reemplaza a las técnicas antiguas basadas en `float`.




### Conceptos clave

Un contenedor flex tiene un **eje principal** (main axis) y un **eje cruzado** (cross axis). Los elementos hijos se distribuyen a lo largo del eje principal.

### Uso básico

```css
.contenedor {
  display: flex;
  flex-direction: row;        /* row (horizontal) | column (vertical) */
  justify-content: center;    /* distribución en el eje principal */
  align-items: center;        /* alineación en el eje cruzado */
  gap: 1rem;                  /* espacio entre elementos */
}

.item {
  flex: 1;                   /* crece para ocupar el espacio disponible */
}
```

### justify-content (eje principal)

| Valor | Efecto |
|---|---|
| `flex-start` | Elementos al inicio (default) |
| `flex-end` | Elementos al final |
| `center` | Centrados |
| `space-between` | Distribuidos, sin espacio en los bordes |
| `space-around` | Distribuidos, con espacio en los bordes |
| `space-evenly` | Distribuidos con espacio igual entre todos |

### align-items (eje cruzado)

| Valor | Efecto |
|---|---|
| `stretch` | Se estiran para llenar el contenedor (default) |
| `flex-start` | Arriba (o izquierda si el eje es column) |
| `flex-end` | Abajo |
| `center` | Centrados verticalmente |

### Ejemplo: centrar un elemento vertical y horizontalmente

```css
.contenedor {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;  /* ocupa toda la pantalla */
}
```

### Ejemplo: navbar horizontal

```css
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: var(--color-secundario);
}
```

### flex-wrap

Por defecto, los items se comprimen para entrar en una línea. Con `flex-wrap: wrap` pueden saltar a la siguiente:

```css
.grilla {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.grilla .card {
  flex: 1 1 300px;  /* crece, se encoge, base mínima de 300px */
}
```

## Diseño Responsive y Media Queries

El diseño responsive permite que una misma página se adapte a diferentes tamaños de pantalla (celular, tablet, desktop).

### Enfoque: Mobile First

Se escriben primero los estilos para **pantalla chica** (celular), y luego se agregan estilos para pantallas más grandes con `@media`:

```css
/* Base: mobile (sin media query) */
.contenedor {
  padding: 1rem;
  flex-direction: column;
}

/* Tablet (768px en adelante) */
@media (min-width: 768px) {
  .contenedor {
    padding: 2rem;
    flex-direction: row;
  }
}

/* Desktop (1024px en adelante) */
@media (min-width: 1024px) {
  .contenedor {
    max-width: 1200px;
    margin: 0 auto;   /* centra el contenedor */
  }
}
```

### Breakpoints comunes

| Nombre | min-width | Dispositivos típicos |
|---|---|---|
| Mobile | — (base) | Celulares |
| Tablet | 768px | Tablets en vertical |
| Desktop | 1024px | Notebooks, monitores |
| Wide | 1280px | Pantallas grandes |

### El meta viewport

Para que el responsive funcione en celulares, es obligatorio incluir en el `<head>`:

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Sin este tag, el celular muestra la versión de escritorio achicada en vez de adaptarse.